

POLITECNICO
MILANO 1863

Software Engineering 2

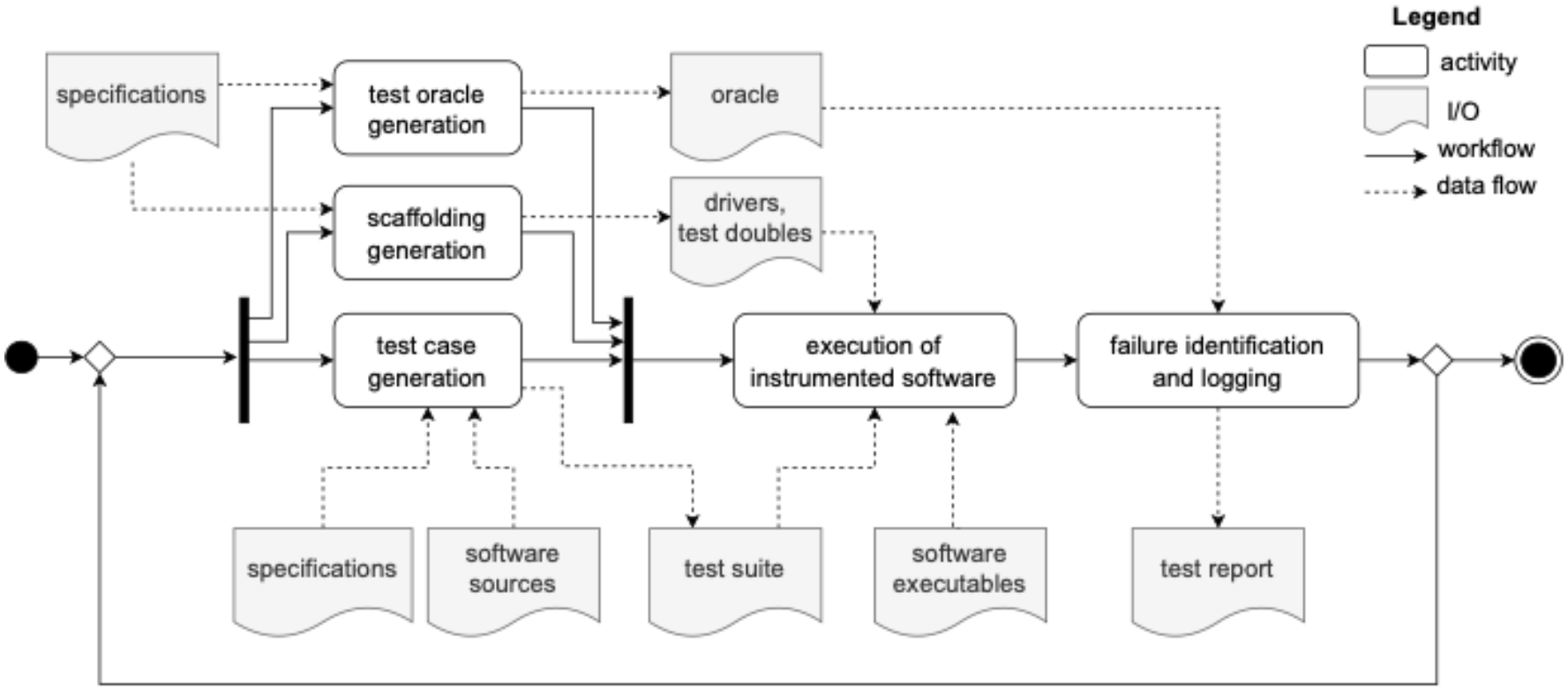
Automated Testing

Concolic Execution

Fuzzing

M Camilli, E Di Nitto, M Rossi

Testing workflow



Test case generation

POLITECNICO MILANO 1863

- Test cases can be defined manually
- Test cases can be automatically generated (automated testing)
  - Combinatorial testing = enumerate all possible inputs following some policy (e.g., smaller to larger).. not in this course!
  - Symbolic execution (we have seen it)
  - Concolic execution = pseudo-random generation of inputs guided by symbolic path properties
  - Fuzz testing (fuzzing) = pseudo-random generation of inputs including invalid, unexpected inputs
  - Search-based testing = explores the space of valid inputs looking for those that improve some metrics (e.g., coverage, diversity, failure inducing capability)

Symbolic execution: test case generation

POLITECNICO MILANO 1863

- **Symbolic execution** can be used to automatically generate test cases
- **Procedure**:
  - Give as input a set of target locations or paths
  - Run symbolic execution
  - If the path condition is SAT for a target location or path
  - Generate one or more input assignments (i.e., test cases) satisfying the path condition
  - Do it for all identified locations and paths to define a test suite
- However, we already discussed the **weaknesses** of symbolic execution…

POLITECNICO
MILANO 1863

Verification & Validation

Automated Testing: Concolic (Concrete-Symbolic) Execution

Concrete-symbolic (concolic) execution

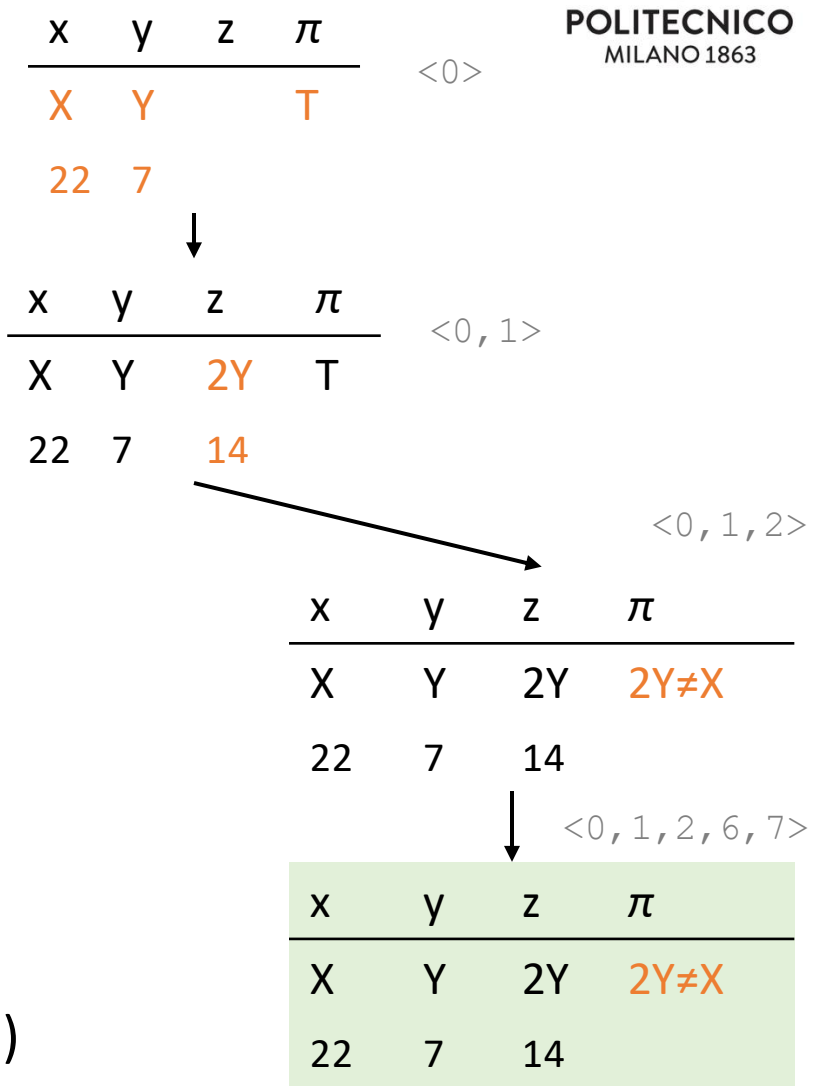
POLITECNICO MILANO 1863

- The very idea
  - Perform symbolic execution **alongside** a concrete one (concrete inputs)
  - The **state** of concolic execution combines a **symbolic** part and a **concrete** part, used as needed to make progress in the exploration
- Steps
  - Concrete to symbolic: derive conditions to explore new paths
  - Symbolic to concrete: simplify conditions to generate concrete inputs

Concolic execution: example (concrete to symbolic)

- **Example**: Let's explore all paths of procedure `m`
- We start from a (random) concrete input, at the same time, we build the symbolic condition of the explored path

```
0: void m(int x, int y) {
1:   int z := 2 * y
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

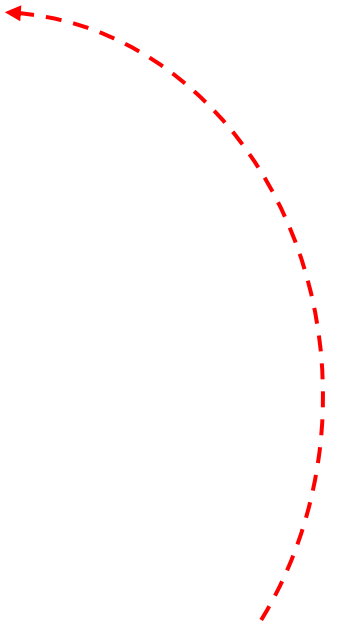


- {x=22, y=7} → path <0,1,2,6,7>, path condition: 2Y≠X
- To explore another path, negate the path condition: ¬(2Y≠X)

Concolic execution: example (symbolic to concrete)

x	y	z	π
X	Y		T
2	1		

<0>



- If we can solve the constraint, we start again with another concrete input that satisfies the new constraint ¬(2Y≠X)

```
0: void m(int x, int y) {
1:   int z := 2 * y
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

Concolic execution: example (concrete to symbolic)

- We explore the new path and apply again the concrete-to-symbolic step

```
0: void m(int x, int y) {
1:   int z := 2 * y
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

- {x=2, y=1} → path <0,1,2,3,4,5,6,7> with path condition 2Y=X ∧ X ≤ Y + 10

<0>

x	y	z	π
X	Y		T
2	1		

<0,1>

x	y	z	π
X	Y	2Y	T
2	1	2	

<0,1,2>

x	y	z	π
X	Y	2Y	2Y=X
2	1	2	

<0,1,2,3>

x	y	z	π
X	Y	Y+10	2Y=X
2	1	11	

<0,1,2,3,4,5,6,7>

x	y	z	π
X	Y	Y+10	2Y=X
2	1	11	X≤Y+10

Partial negation: last condition only

2Y=X ∧ ¬(X ≤ Y + 10)

Concolic execution: example (symbolic to concrete)

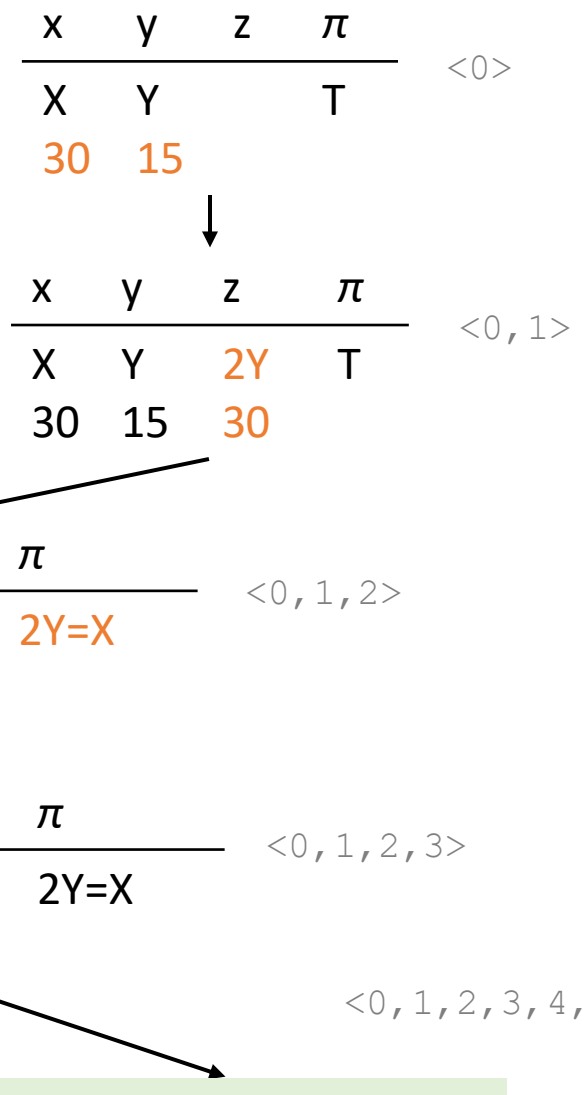
x	y	z	π
X	Y		T
30	15		

- New input values are identified

```
0: void m(int x, int y) {
1:   int z := 2 * y
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

Solve: $2Y=X \wedge \neg(X \leq Y + 10) \equiv$
$2Y=X \wedge X > Y + 10$

Concolic execution: example (concrete to symbolic)



- We explore the new path and apply again the concrete-to-symbolic step

```
0: void m(int x, int y) {
1:   int z := 2 * y
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

- Conclusion: we have been able to cover all paths with the following test cases:
  - <0,1,2,6,7>: {x=22, y=7}
  - <0,1,2,3,4,5,6,7>: {x=2, y=1}
  - <0,1,2,3,4,6,7>: {x=30, y=15}

Concolic execution: example2

x	y	z	π	
X	Y		T	<0>
22	7			

↓

x	y	z	π	
X	Y	bb(Y)	T	<0,1>
22	7	14		

<0,1,2,6,7>

x	y	z	π
X	Y	bb(Y)	bb(Y)≠X
22	7	14	

- **Example**: Let's explore again all paths of the procedure `m2`

```
0: void m2(int x, int y) {
1:   int z := bb(y) //black-box function
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

- We try to follow the same approach, but, in some cases, we cannot solve the symbolic condition…
- Behavior of `bb` is unknown. We execute it with the identified input cases
  - Example: run bb(7) returns 14
- Now the condition can be solved

Negate: ¬(bb(Y)≠X)

Solve: ¬(bb(Y)≠X) → cannot do it symbolically!

Symbolic-to-concrete step

¬(14≠X) ≡ 14=X

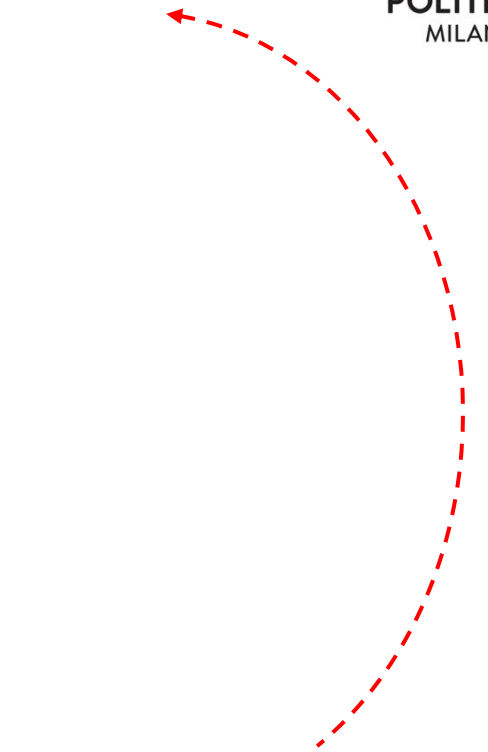
Concolic execution: example2

x	y	z	π
X	Y		T
14	7		

<0>

- Now the constraint can be solved and we can start a new exploration

```
0: void m2(int x, int y) {
1:   int z := bb(y) //black-box function
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```



Concolic execution: example2

- New explorations follow the same approach

```
0: void m2(int x, int y) {
1:   int z := bb(y) //black-box function
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

x	y	z	π	
X	Y		T	<0>
14	7			

↓

x	y	z	π	
X	Y	bb(Y)	T	<0,1>
14	7	14		

x	y	z	π	
X	Y	bb(Y)	bb(Y)=X	<0,1,2>
14	7	14		

↓

x	y	z	π	
X	Y	Y+10	bb(Y)=X	<0,1,2,3>
14	7	17		

<0,1,2,3,4,5,6,7>

x	y	z	π
X	Y	Y+10	bb(Y)=X
14	7	17	X≤Y+10

Negate last condition:
bb(Y)=X ∧ ¬(X≤Y+10) ≡ bb(Y)=X ∧ X>Y+10

Solve: bb(Y)=X ∧ X>Y+10 → cannot!

Concolic execution: example2

x	y	z	π	
X	Y		T	<0>
34	17			

- New explorations follows the same approach

```
0: void m2(int x, int y) {
1:   int z := bb(y) //black-box function
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

Concretize: bb(Y)=X ∧ X>Y+10
We select Y randomly, execute bb(Y) and check if the formula holds
Example: Y=17, bb(Y)=34
Solve: Y=17 ∧ bb(Y)=34 ∧ bb(Y) =X ∧ X>Y+10

Concolic execution: example2

- New explorations follows the same approach

```
0: void m2(int x, int y) {
1:   int z := bb(y) //black-box function
2:   if (z == x) {
3:     z := y + 10
4:     if (x <= z)
5:       print("Log message.")
6:   }
7: }
```

x	y	z	π	
X	Y		T	<0>
34	17			

↓

x	y	z	π	
X	Y	bb(Y)	T	<0,1>
34	17	34		

↓

x	y	z	π	
X	Y	bb(Y)	bb(Y)=X	<0,1,2>
34	17	34		

↓

x	y	z	π	
X	Y	Y+10	bb(Y)=X	<0,1,2,3>
34	17	27		

<0,1,2,3,4,6,7>

x	y	z	π
X	Y	Y+10	bb(Y)=X
34	17	27	X>Y+10

Concolic execution: pros and cons

POLITECNICO MILANO 1863

- Advantages
  - Can deal with black-box functions in path conditions (not possible with symbolic exec)
  - Can generate test cases automatically
- Limitations
  - Finds just **one input example** per path, however…
    - Failures typically occur with certain inputs only
    - If failures are rare events, it's unlikely to spot them with concolic exec
  - #paths explode due to complex nested conditions → **large search space**
  - **Does not guide the exploration**, it just explores possible paths one by one as long as we have budget (e.g., time, #runs)

POLITECNICO
MILANO 1863

Verification & Validation

Automated Testing: Fuzz Testing (fuzzing)

Fuzzing, motivations

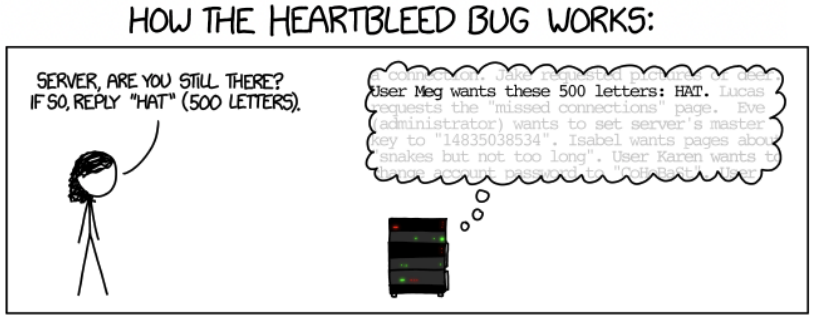
POLITECNICO MILANO 1863

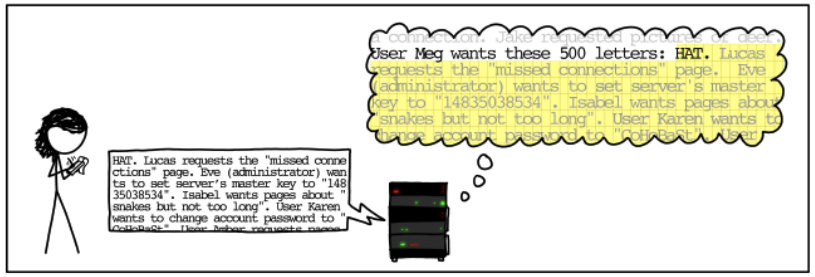
- Complements functional testing (e.g., manually defined test cases based on functional requirement specs)
  - Works at **component** or **system** level
  - Deals also with external qualities other than correctness, e.g., **reliability** and **security** by providing **randomly** generated data as inputs
  - Can uncover defects that might not be caught by other methods since **randomness** typically leads to **unexpected**, or **invalid** inputs
- Essence of fuzzing
  - Create random inputs, and see if they break things
  - Let it run long enough, and you'll see

HeartBleed bug, a famous example

https://heartbleed.com/

- Security bug in OpenSSL library (cryptographic protocols for remote communication)
- Unchecked memory access
  - Exploited by sending a specially crafted command to the SSL heartbeat service
  - Introduced in 2012 → discovered and fixed in 2014
  - Detected by researchers at Google using fuzzing + runtime memory-checks





Continue…
Full strip:
https://xkcd.com/1354/

Fuzzing, an example

POLITECNICO MILANO 1863

- Let's assume we want to **fuzz an existing program**, such as `bc`
  - `bc` is a UNIX utility "basic calculator"
  - It expects as input a stream of chars representing mathematical expressions
    - Example: `echo "(1+3)*2" | bc` returns `8`
- We evaluate the robustness of `bc` given an unpredictable input stream following these steps:
  - **Build a fuzzer**: a program that will output a random char stream
  - **Attack** `bc` using the fuzzer with the goal of trying to break it

A simple fuzzer

- Let's build a simple fuzzer (fuzz generator)..
  - We use Python (you can see it as pseudocode if it is not familiar)

```
# A string of up to `max_length` characters in the range
#  [`char_start`, `char_start` + `char_range`)

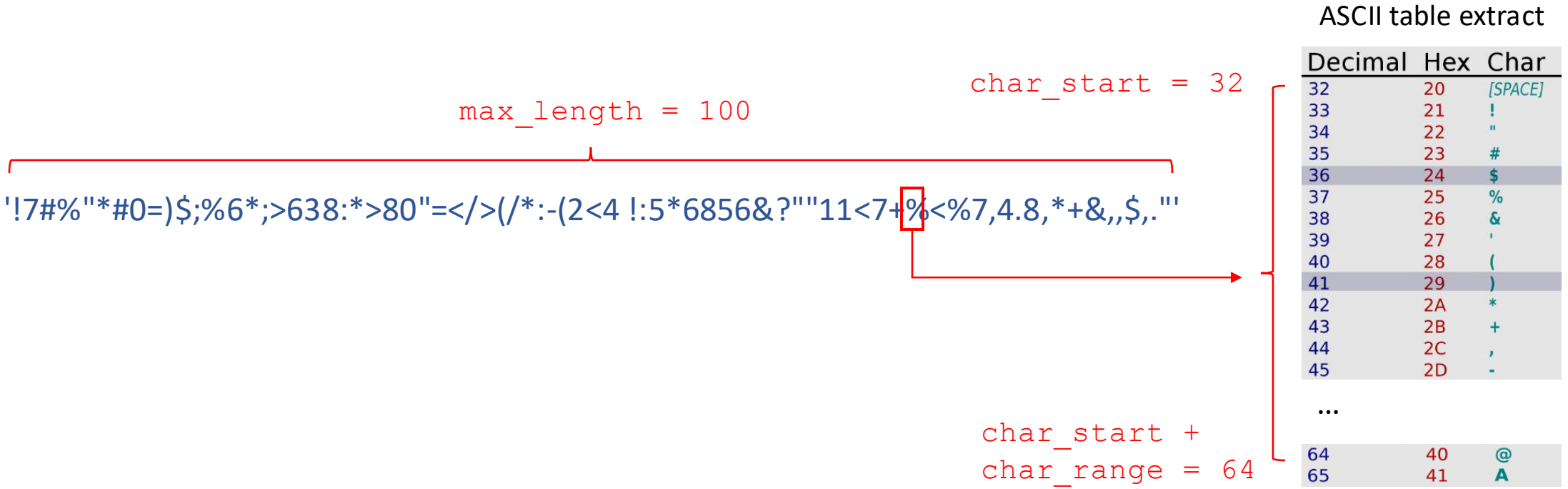
def fuzzer(max_length: int = 100, char_start: int = 32, char_range: int = 32) -> str:
    string_length = random.randrange(0, max_length + 1)
    out = ""
    for i in range(0, string_length):
        out += chr(random.randrange(char_start, char_start + char_range))
    return out
```

random integer in [0, max_length]

chr(*n*) returns the character with ASCII code *n*

A simple fuzzer

- What is a possible output of `fuzzer()` with default args?



A simple fuzzer

POLITECNICO MILANO 1863

- Fuzz input = term for such random, unstructured data
- Assume a program expects a specific input format (e.g., comma-separated list of values, e-mail address)
  - What happens if we give fuzz strings as input?
  - Can the program process such an input without any problems?
- Note: fuzzers can produce any kinds of input, for example:
  - sequence of lowercase letters (up to 100)
  - up to 1000 random digits
  - up to 200 alternating lowercase and uppercase letters
  - Other data types: integer, float, …
  - **Exercise**: use and/or modify `fuzzer()` to produce these fuzz inputs

Fuzzing external programs

POLITECNICO MILANO 1863

- To test the program we create an input file (test data); then we feed this input file to `bc`
- Here is an example of manually defined test case

```
import os
import tempfile, subprocess

basename = "input.txt"
tempdir = tempfile.mkdtemp()
FILE = os.path.join(tempdir, basename) # tmp file s.t. we do not clutter the file system

program = "bc" # simple unix calc utility
input = "2 + 2\n" # nominal input
with open(FILE, "w") as f:
    f.write(input)
result = subprocess.run([program, FILE],
                        stdin=subprocess.DEVNULL,
                        stdout=subprocess.PIPE,
                        stderr=subprocess.PIPE,
                        universal_newlines=True)
```

Note:
executes an external program, similar to
`Runtime.getRuntime().exec(…)` in Java
`system(…)` in C

Fuzzing external programs

- We can use the fuzzer and pass a large number of inputs to `bc`, to see whether it might crash on some
- **Note**: running this may take a while.. so, we need to define a reasonable **budget**

```
trials = 100 # testing budget
program = "bc"

runs = []
for i in range(trials):
    with open(FILE, "w") as f:
        data = fuzzer()
        f.write(data)
    result = subprocess.run([program, FILE],
                            stdin=subprocess.DEVNULL,
                            stdout=subprocess.PIPE,
                            stderr=subprocess.PIPE,
                            universal_newlines=True)
    runs.append((data, result))
```

As long as we have budget

At the end runs contains all generated inputs with corresponding results

Fuzzing external programs

POLITECNICO MILANO 1863

- From the result, we can check the program output and the status

```
result.stdout # e.g., '4\n'
result.returncode # e.g., 0 if terminated correctly
result.stderr # e.g., '' if no error msgs
```

- After the execution, we can query the `runs` structure (i.e., our result log)
  - How many runs passed? (no error messages)

```
sum(1 for (data, result) in runs if result.stderr == "")
```

  - how many crashed?

```
sum(1 for (data, result) in runs if result.returncode != 0)
```

Fuzzing external programs — Warning

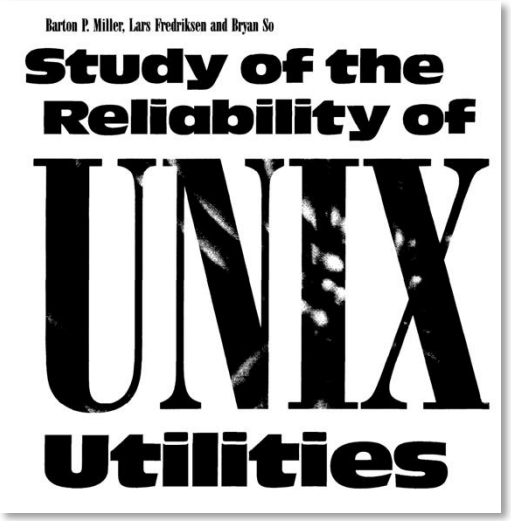
POLITECNICO MILANO 1863

- Note: instead of `bc`, you can put in any program (e.g., any UNIX utility)
- Be aware that if the program can change your system, fuzz inputs **may cause damages**!
  - **Example**: let's imagine we test: `rm -fr FILE`
  - **Exercise**: What is the chance of causing damages?
    - e.g., "`/<white space><other>`" will erase your entire file system
      e.g., "`~<white space><other>`" will erase your home dir
      e.g., "`.<white space><other>`" will erase the current folder

Fuzzing external programs

- [Miller et al., 1990] About a third of the UNIX utilities (including the *bc* utility) they fuzzed had issues – crash, or hang when tested with fuzz inputs
- Apparently, the bugs have been fixed.. you can try to replicate the experiment!

Barton Miller et al., 1990. An empirical study of the reliability of UNIX utilities. Commun. ACM 33, 12 (Dec. 1990), 32–44.
https://doi.org/10.1145/96267.96279
Assignment Miller gave to his students to build the research:
https://pages.cs.wisc.edu/~bart/fuzz/CS736-Projects-f1988.pdf



Common bugs found by fuzzers

POLITECNICO MILANO 1863

- Buffer overflow
  - Many programs have built-in maximum lengths for inputs
  - In languages like *C*, it is easy to exceed these lengths, thus causing buffer overflows

```
char weekday[9];  // 8 characters + trailing '\0' terminator
…
strcpy(weekday, input);
```

What if `input` is "Wednesday"?

"\0" is copied in memory after `weekday`
→ it may overwrite other vars causing arbitrary behavior

Common bugs found by fuzzers

POLITECNICO MILANO 1863

- Missing error check
  - Many languages, like *C*, do not have exceptions, instead they have functions that return special error codes in exceptional circumstances:

```
/* getchar returns a character from stdin; if no input is available
anymore, it returns EOF */

while (getchar() != ' ');
```

Here the program reads the input, char by char until a whitespace (`' '`) occurs…
what if the input ends (`EOF`) before a whitespace?

`getchar()` returns `EOF`, and keeps on returning `EOF` when called again
→ the code enters an infinite loop (hangs)

Common bugs found by fuzzers

POLITECNICO MILANO 1863

- Rogue numbers
  - Fuzzing easily generates uncommon values in the input, causing interesting behavior

```
/* reads size from the input, and then allocates a buffer of the given size */

char* read_input() {
    size_t size = read_size();
    char *buffer = (char *)malloc(size);
    // fill buffer
    return buffer;
}
```

What if `size` is very large, exceeding program memory? What if `size` is less than the number of characters in `stdin`? What if `size` is negative?

→ by providing a random number for `size`, fuzzing can create all kinds of damages

Checking memory accesses

POLITECNICO MILANO 1863

- Best practice: fuzzing + runtime memory-checks
  - Catch problematic memory accesses during testing by instrumenting programs with memory-checking environments
    - Example: *C* lang Address Sanitizer https://clang.llvm.org/docs/AddressSanitizer.html
  - Instrumentation checks at runtime every memory operation to detect potential violations
    - Out-of-bounds accesses to heap, stack
    - Use-after-free
    - Double-free
  - Cost-effectiveness trade-off
    - Increases execution time (typical slowdown is 2x and more memory)
    - Decreases human effort to find these bugs

Fuzzing with mutations: problem

POLITECNICO MILANO 1863

- Problem
  - Many programs expect inputs in very specific formats before they would actually process them
  - In this case, completely random inputs have low chance to execute deep paths
- Example
  - Assume we have a program that accepts a URL
  - What is the chance of producing a valid URL when fuzzing with random inputs?

Fuzzing with mutations: problem

POLITECNICO MILANO 1863

- More details about the example…
- A URL has a number of elements: `scheme://netloc/path?query#fragment`
  - `scheme` is the protocol (http, https, …)
  - `netloc` is the host name (e.g., www.google.com)
  - `path` is the path on that host (e.g., search)
  - `query` is a list of key-value pairs (e.g., q=fuzzing)
  - `fragment` is a marker for a location in the retrieved document (e.g., #result)
- Let's use `fuzzer(char_start=32, char_range=96)`
  - we use the full range of printable ASCII characters (including `:` and `/`)
  - we need a string starting with "http://" or "https://"

Fuzzing with mutations: problem

POLITECNICO MILANO 1863

- Let's use `fuzzer(char_start=32, char_range=96)`
  - we need a string starting with "http://" or "https://"
- What is the chance?
  - We have two sequences of 7 and 8 very specific characters
    - $(1/96)^7 + (1/96)^8 = 1.3446 * 10^{-14}$ (likelihood)
- What is the time required to produce a valid URL?
  - Assume a single run is very fast, say, ~1 microsec?
    - $1/\text{likelihood} = 7.4370 * 10^{13}$ (avg #runs)
    - $10^{-6} * \text{avg \#runs} = 7.430 * 10^7$ seconds ≈ 20658 hrs ≈ 860 days ≈ 2.4 years

Fuzzing with mutations: the idea

POLITECNICO MILANO 1863

- In this case we wait 2.4 years on average to get a single valid URL
  - → good chance of finding bugs in input parsing
  - → little chance of reaching any deeper functionality
- The generation should be guided!
- Mutational fuzzing: rather than random inputs from scratch (generational fuzzing), we mutate a given valid input
  - Mutation = simple input manipulation
    - e.g., with strings we can insert a character, delete a character, or flip a bit in character representation

Fuzzing with mutations: example

```
import random

def insert_random_char(s: str) -> str:
    pos = random.randint(0, len(s))
    random_char = chr(random.randrange(32, 127)) // rand printable char
    return s[:pos] + random_char + s[pos:]
…

def mutate(s: str) -> str:
    mutators = [ delete_random_char,
                 insert_random_char,
                 flip_random_char ]
    mutator = random.choice(mutators)
    return mutator(s)
```

Implementation of mutation operators (mutators)

Generates a fuzz input by applying a random mutator

Fuzzing with mutations: example

POLITECNICO MILANO 1863

- We can even apply multiple mutations, e.g., 20 or 50 mutations

```
seed_input = "http://www.google.com/search?q=fuzzing"
mutations = 50
fuzz_input = seed_input
for i in range(mutations):
    print("{} mutations: {}".format(i, fuzz_input))
    fuzz_input = mutate(fuzz_input)
```

```
0 mutations: http://www.google.com/search?q=fuzzing
…
10 mutations: http:/L/www.ggoWglej.com/seaRchqfu:in
…
30 mutations: htv://>fwggoVgle"j.qom/ea0Rd3hqf,u^:i
…
50 mutations: htv://>fwgeo]6zTle"BjM.\'qom/eaR[3hqf,tu^:i
```

Observations

- Multiple mutations
  → higher variety of inputs
- Too many mutations
  → higher chance of invalid inputs

Fuzzing: guiding by coverage

POLITECNICO MILANO 1863

- The higher the variety → the higher the risk of an invalid input
- We should keep and mutate inputs that are especially valuable
  - Guiding by **coverage** information is a popular approach
    - Coverage (line, branch, or path) is a common metric for test quality
    - In this case we say the test case generation is **gray box**

- The very idea
  - Evolve only those test cases that have been successful
    - Success = found a new path
  - Fuzzer keeps and maintains a population of successful inputs; if a new input finds another path, it will be retained as well

EXTRA MATERIAL

Fuzzing: guiding by coverage

POLITECNICO MILANO 1863

- We define a utility function to run test cases
  - We assume there exists a `Coverage` class that we can use to retrieve coverage of a test run

```
PASS = "PASS"
FAIL = "FAIL"

def run_function(foo: Callable, inp: str) -> Any:
    with Coverage() as cov:
        try: result = foo(inp)
             outcome = PASS
        except Exception:
            result = None
            outcome = FAIL
    return result, outcome, cov.coverage()
```

The `coverage()` method returns the coverage achieved in the last run as list of tuples:
<function-name, executed-line number>

```
[ ('urlsplit', 465),
  ('urlparse', 394),
  ('urlparse', 400) ]
```

#lines executed in the last run

EXTRA MATERIAL

Fuzzing: guiding by coverage

POLITECNICO MILANO 1863

```
def mutation_coverage_fuzzer(foo: Callable, seed: List[str], min_muts: int = 2,
        max_muts: int = 10, budget: int = 100) -> List[Tuple[int, str, int]]:
    population = seed
    cov_seen = set()
    summary = []
    for j in range(budget):
        candidate = random.choice(population)
        trials = random.randint(min_muts, max_muts)
        for i in range(trials):
            candidate = mutate(candidate)
        result,outcome,new_cov = run_function(foo, candidate)
        if outcome == PASS and not set(new_cov).issubset(cov_seen):
            population.append(candidate)
            cov_seen.update(new_cov)
            summary.append((j, candidate, len(cov_seen)))
    return summary
```

create candidate test case

run test case with coverage

evaluate "goodness" of test case

Set union

EXTRA MATERIAL

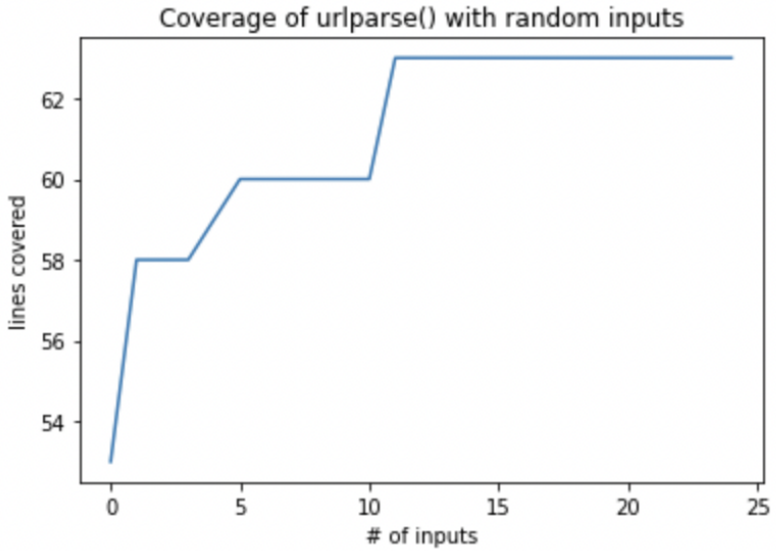
Fuzzing: guiding by coverage

POLITECNICO MILANO 1863

- Let's test a target program `urlparse` with 10k fuzz inputs

```
seed_input = "http://www.google.com/search?q=fuzzing"
summary = mutation_coverage_fuzzer(urlparse, [seed_input], budget = 10000)
```

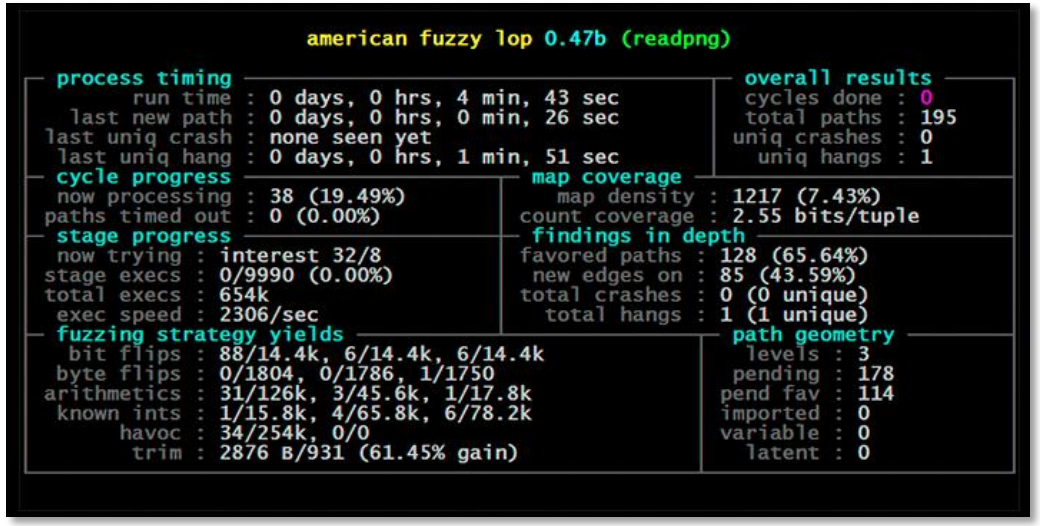
- By inspecting `summary` we can see each and every input is valid and has different coverage!
- Strengths
  - Practical also with large programs – it explores one path after the other until you have budget
  - All you need is a way to capture the coverage



AFL, a succesful story

- [Nov 2013] 1st version of American Fuzzy Lop (AFL) was released, one of the most successful fuzzing tools
  - https://lcamtuf.coredump.cx/afl/

- First to demonstrate that failures and vulnerabilities can be detected automatically in large-scale security-critical applications
- Implements mutational fuzzing + guiding by coverage



AFL User Interface

Positioning fuzzing in the testing workflow

Fuzzing introduces automation here.
It is gray box as it uses coverage information

